working apps, missing checks

an app can let people sign in, publish a page and reach checkout while still accepting actions they should never be allowed to perform. the important question is where those permissions are enforced and who checks them before release.

this part reviews published research. the screenshots come from the original disclosures.

what the cases show

case	failure	type of evidence
linkable / 2025 [1, 2]	unauthorized writes included a paid status	researcher demonstration; disputed cve
moltbook / 2026 [3]	exposed tokens and unauthorized post edits	researcher demonstration
base44 / 2025 [4]	private-app registration restrictions bypassed	vendor-confirmed and fixed
escape / 2025 [5]	weak backend access across a public app sample	deployment investigation
wordpress addon / 2023 [6, 7]	password reset without proving account control	disclosed flaw and observed attacks
android generators / 2018 [8]	shared generated code and configuration weaknesses	peer-reviewed technical study
mobile firebase apps / 2022 [9]	backend data left publicly accessible	deployed exposures; build method unknown

the comparison

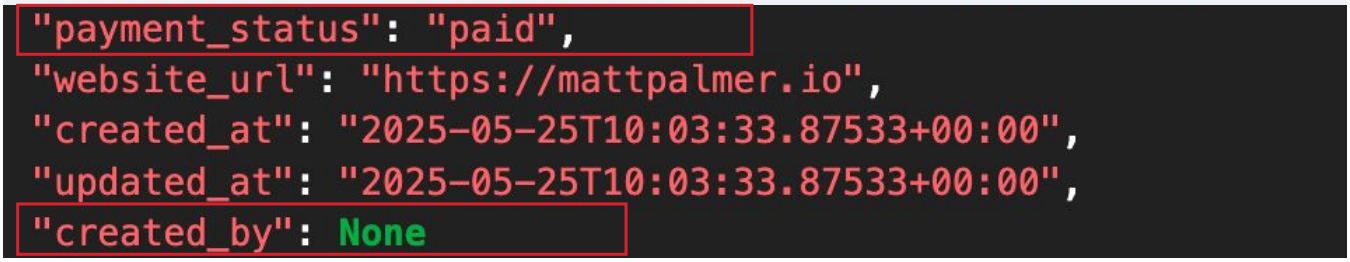
ai can write the code. a visual builder can assemble existing components. a mobile generator can reuse a shared app template. a developer can write or copy the same logic manually. these approaches can overlap in one product.

the cases support checking backend permissions, shared components and release settings in each approach. they do not provide a fair overall ranking of ai, builders and human developers.

linkable / payment state

matt palmer first reported access to linkable's users table in march. in a may follow-up, he documented reads and writes after removing the authorization header. the changed identity context received broader access than it should have. [1]

linkable



matt palmer / published payload [1]

the disclosure documents insertion of a record marked as paid. this shows manipulation of the application's payment state; it does not show a compromise of stripe's own payment system. [1]

step	published result	missing check
direct database access	user data became readable	access restricted to the intended caller
authorization header removed	unauthorized writes became possible	separate rules for the anonymous role
paid state submitted	a paid record was inserted	only trusted payment processing may set this state

why it worked

the database allowed actions that the app's ownership and payment rules should have rejected. a working checkout did not protect the payment field. [1]

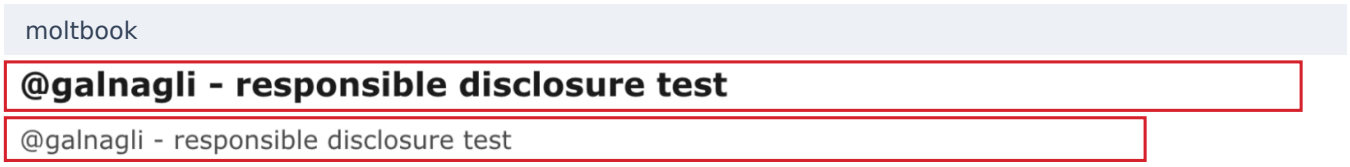
what i would change

deny anonymous access to private records and test each database operation separately. keep payment state outside client-writable fields. let a backend verify the payment event, match it to the order and apply the transition once. stripe documents signature verification and safe handling of repeated event deliveries. [11] [12]

cve-2025-48757 covers the broader lovable rls issue. the supplier disputes the assignment because customers are responsible for protecting their application data. [2]

moltbook / ownership

wiz found exposed agent tokens, emails and private messages in moltbook's supabase database, including about 1.5 million authentication tokens. its founder described building the platform with ai. [3]



wiz / post modified during disclosure [3]

wiz changed an existing post without authorization. an initial read-access fix left post writes open, requiring another fix. the image shows the researchers' demonstration. [3]

boundary	observed failure	effect
credentials	tokens readable	impersonation possible
ownership	existing posts editable	content integrity lost
fix coverage	reads fixed before writes	another operation remained exposed

why it worked

the public client key was expected. backend permissions failed to enforce who could read or change each record. [3]

what i would change

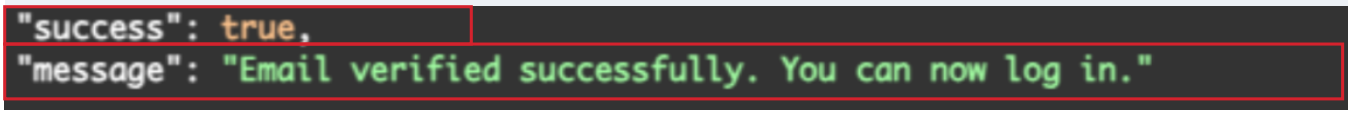
keep authentication tokens out of publicly readable tables. define ownership rules for reads, inserts, updates and deletes, including the anonymous role. after a fix, check the other tables and operations as well. privileged database keys must stay on the backend. [11]

wiz documents researcher access and changes. it does not establish a criminal campaign or identify the individual instruction that caused this configuration.

base44 / private access

wiz registered for a private sso application using its public app id. the researchers completed email verification and gained access despite the intended restriction. [4]

base44



wiz / published verification response [4]

step	what happened	security decision that failed
app id	publicly available	not a secret credential
registration	outsider registered and verified	private-app policy bypassed
access	new account gained entry	email verification was not approved membership

why it worked

the shared authentication flow omitted the app's privacy policy. customer apps inherited that platform decision. [4]

wix fixed the issue within 24 hours of the july 9 report, and wiz verified the fix. wix reported no evidence of prior abuse. this case establishes a platform authentication flaw; it does not establish that ai generated the vulnerable handler. [4]

what i would change

apply the same membership policy to registration, invitation, verification, login and account recovery. a private application should require an approved identity or invitation through every entry path. include direct api calls in the release checks.

for a hosted builder, the vendor owns the shared fix. the application owner still needs to verify the privacy settings and understand what the platform actually enforces.

how broad is the problem

escape / deployed builder applications

escape's october 2025 methodology describes more than 5,600 public applications, over 2,000 vulnerabilities, 400-plus exposed secrets and 175 instances of exposed personal information. these are different measures, not counts of confirmed breached companies. [5]

part of the method	what it adds	limit
public directories and fingerprints	real deployments	discovery bias
missing tokens or changed headers	access restrictions checked	not evidence of criminal use
manual checks on a subset	automated findings reviewed	uneven manual coverage

the dataset was heavily weighted toward lovable and collected as a snapshot. it supports a pattern of deployed access-control problems. it cannot establish a representative failure rate for every builder or compare builders fairly with human-written applications. [5]

perry et al. / developers using an assistant

a ccs 2023 study used 47 participants and five security-related programming tasks. participants with an assistant produced insecure solutions more often on four of the five tasks. they were also more likely to believe their code was secure. the study used the older codex-davinci-002 model. [10]

this is controlled evidence about a particular study setting, rather than a measurement of today's ai products or deployed business applications. it gives a reason to verify generated solutions and confidence in them; it does not settle the whole comparison. [10]

what i take from both

working output and developer confidence are weak substitutes for checking the backend. a useful comparison needs the same requirements, the same attack cases and the same review effort for every development approach.

wordpress / a ready-made form

this is a non-ai example from a visual page-builder ecosystem. essential addons for elementor 5.4.0 through 5.7.1 let an unauthenticated caller reset another user's password when the username was known. the handler failed to validate the password-reset key. [6]

essential addons

```
$user = check_password_reset_key( $rp_data_db['rp_key'], $rp_data_db['rp_login'] );
```

patchstack / added reset-key validation [6]

feature or check	why it was insufficient	result
recovery form	missing reset-key validation	unauthorized password change
frontend nonce	available to visitors	not proof of account control
shared plugin	same handler across sites	shared vulnerability

patchstack disclosed the issue in may 2023; version 5.7.2 was the first fix. wordfence then reported more than 6,900 blocked exploit attempts over five days. those are observed attack attempts, not a count of successful takeovers. [6] [7]

why it matters here

the site owner could add the form without writing its authentication code. security still depended on the plugin's handler, creating a dependency to review and update.

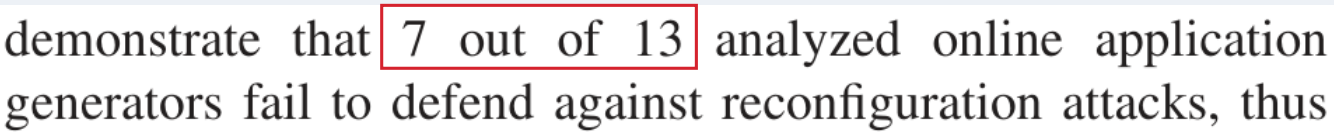
what i would change

keep the plugin on a maintained release with current fixes. validate the reset token for the target account before changing a password, and check permissions for other privileged actions. wordpress explicitly says nonces do not replace authentication or authorization. [15]

android / shared generated code

oltrogge and colleagues studied android generators and a corpus of about 2.29 million free apps. at least 11.1% were generated. seven of thirteen tested generators were susceptible to reconfiguration attacks. [8]

android app generators



oltrogge et al. / published finding [8]

problem in the study	how it could be abused	where it came from
unprotected configuration	change modules or content	app and update path
unsafe webview behavior	reach native features	shared bridge code
exposed components	reach internal functionality	generated android components

andromo and app inventor generated code for selected modules and had fewer issues. they avoided the configuration attack described for monolithic generators. these are 2018 findings, not current-version claims. [8]

why it matters here

a website-to-app wrapper adds a native application around web content. that wrapper may introduce permissions, exported components and bridges the website did not have. the same template can carry those decisions into many apps.

what i would change

review the built apk and its backend. allow only intended web origins, reject invalid tls certificates and remove sensitive native bridges from untrusted content. limit permissions and keep access checks on the backend. [14]

firebase / deployment settings

check point identified 2,113 mobile applications with exposed firebase databases in a virustotal-based investigation. reported examples included private messages, credentials and location data. the researchers reached the data through the database endpoint. [9]

firebase

```
{
  "rules": {
    ".read": true,
    ".write": true
  }
}
```

check point / example of open test-mode rules [9]

the report describes developers opening database access for testing and leaving it open when releasing the app. it does not identify the affected apps as ai-generated or built with a visual builder. [9]

policy	what it actually proves	remaining risk
public reads and writes	any reachable client can access data	no identity or ownership boundary
any signed-in user may access everything	the caller has an account	one customer can still reach another's data
identity plus record ownership	the caller is permitted to access this record	also validate fields and privileged actions

what i would change

start with denied access, then add the operations each user needs. bind private records to the verified user id and protect ownership fields from being changed. firebase warns that requiring a login alone can still give every account access to the whole database. [13]

check point's cases show the deployment problem. the same backend checks should apply whether the mobile interface was written manually, assembled with blocks or generated from a prompt.

what i would require before release

i would compare the backend decisions under the same conditions. the app should keep these rules even when the normal interface is skipped or a client changes its request.

area	required behavior	check before release
private accounts	registration, login and recovery enforce the same membership policy	try direct api access as an uninvited user
ownership	a verified identity can access only the records its role permits	try another user's record for every operation
payment state	only trusted processing can mark an order paid	reject client-supplied paid state and replayed events
database access	private data is denied to anonymous and unrelated users	test missing, expired and valid sessions separately
plugins and generated apps	the released component and its update path are reviewed	inspect the deployed site or apk, not only its project editor
maintenance	someone owns fixes, credentials and release settings	repeat the checks after changes and dependency updates

database checks should cover each operation and role. payment updates should use verified events and handle repeated deliveries. mobile wrappers need their own origin and bridge review. [11] [12] [13] [14]

who owns the missing work

approach	work that still needs an owner
ai-generated code	review the proposed design, generated logic and deployed database policy
visual or hosted builder	check platform access rules, installed components and vendor fixes
mobile generator	review the compiled app, shared template and backend together
manual development	review copied code, permission checks, shortcuts and release settings

lovable's current documentation describes built-in security scans and still calls for a thorough review. the 2025 disclosures should not be read as a description of every current platform feature. [16]

my conclusion is that producing a working app is only one part of the job. the security claim needs evidence that unauthorized identities and invalid business actions are rejected.

original disclosures, research papers and official guidance

references

[1] matt palmer / linkable disclosure
march and may 2025 findings; the inserted record is in appendix a2

[2] cve record / cve-2025-48757
the supplier disputes the assignment on customer-responsibility grounds

[3] wiz / moltbook
february 2026 disclosure, exposed credentials and demonstrated post changes

[4] wiz / base44
july 2025 registration bypass, vendor response and verified fix

[5] escape / investigation methodology
october 2025 dataset, validation process and sampling limits

[6] patchstack / essential addons for elementor
original analysis and patch for cve-2023-32243

[7] wordfence / observed exploit attempts
may 2023 attack telemetry for the same wordpress vulnerability

[8] oltrogge et al. / online app generators
ieee security and privacy 2018; sections v-vi and the discussion

[9] check point research / exposed mobile databases
march 2022 firebase investigation; 2,113 mobile applications

[10] perry et al. / ai-assisted programming study
ccs 2023; 47 participants, five tasks and an older codex model

[11] supabase / row-level security
database roles, ownership policies and privileged keys

[12] stripe / webhook verification
verify event signatures and handle duplicate deliveries safely

[13] firebase / insecure security rules
open access, overly broad authenticated access and owner checks

[14] android developers / webview native bridges
trusted origins, secure transport and restricted native interfaces

[15] wordpress / nonce security
a nonce does not replace authentication or permission checks

[16] lovable / current security documentation
built-in scans support review; application requirements remain the owner's responsibility